

CASE STUDY · FULL-STACK DEVELOPMENT

ShopEase

A Production-Style Multi-Vendor E-Commerce Platform
Built on the MERN Stack

Muhammad Haroon Khan

BSCS (7th Semester), University of Mianwali · Dev Weekends '26 Fellowship
September 2026

1. Overview

ShopEase is a full-stack, multi-vendor e-commerce marketplace where independent sellers register, get approved, and manage their own storefronts, while buyers browse products from many sellers in one unified shopping experience. It was built end-to-end — authentication, product management, a real Stripe-powered checkout, a multi-vendor order system, seller and admin dashboards, and a review system — as the flagship assignment of the Dev Weekends '26 fellowship.

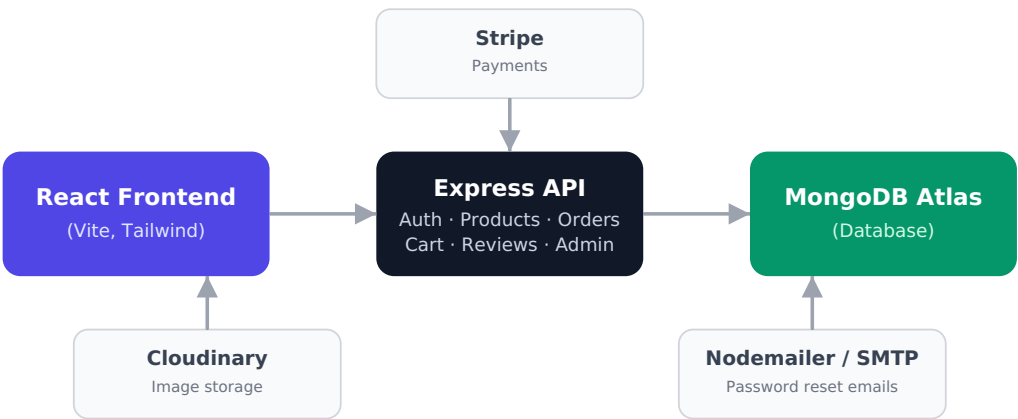
2. Goals of the Project

- Build a genuinely functional multi-role authentication system (buyer, seller, admin) — not just a UI mockup
- Implement a real, working payment flow using Stripe's test-mode API, not a simulated checkout
- Correctly handle the hardest part of multi-vendor commerce: splitting one cart into per-seller orders
- Give sellers and admins dashboards with real operational value
- Produce a portfolio-quality deliverable: a live deployed URL, a clean GitHub repository, and this case study

3. System Architecture Overview

The application follows a decoupled MERN architecture: a React (Vite) single-page frontend talks to an Express/Node.js REST API, backed by MongoDB Atlas for persistence. Three external services are integrated directly into the backend: **Stripe** for payments, **Cloudinary** for product image storage, and **Nodemailer** (SMTP) for transactional password-reset emails.

4. System Architecture Diagram



5. Key Features

5.1 Multi-Role User System

Role	Capabilities
Buyer	Browse/search products, manage cart, checkout via Stripe, track orders, leave verified-purchase reviews
Seller	Register with a shop profile (pending admin approval), list/edit/delete products with image upload, manage incoming orders
Admin	Approve/revoke sellers, view all products and orders platform-wide, view aggregate revenue and growth stats

5.2 Product Management & Shopping

- Product listings with multi-image upload (Cloudinary), stock tracking, and optional discount pricing
- Persistent server-side cart tied to the buyer's account, with quantity controls
- Keyword search and category filtering, with server-side pagination

5.3 Payment Processing

- Real Stripe integration (test mode) — a single, well-integrated gateway rather than several shallow ones, so the flow could be built properly end-to-end
- Secure transactions: the server independently re-verifies each PaymentIntent's status with Stripe before ever creating an order — client-reported success is never trusted alone

Support for additional gateways (e.g. PayPal, local payment rails) is a natural next step — see Future Improvements.

5.4 Seller Dashboard Features

- **Order management:** sellers see only their own incoming orders and can move each through Processing → Shipped → Delivered/Cancelled
- **Revenue & platform stats:** aggregate totals (orders, revenue, buyer/seller counts) are available on the Admin dashboard today; a dedicated per-seller analytics view (sales-over-time charts) is scoped as a future enhancement rather than built in this version

6. API Architecture

A resource-oriented REST API, organized by domain, with role-based middleware guarding each protected route:

Resource	Purpose
/api/auth	Register, login, logout, forgot/reset/change password
/api/products	Public browsing/search + seller-only create/update/delete
/api/cart	Buyer-only cart management
/api/payments	Stripe PaymentIntent creation
/api/orders	Checkout, buyer order history, seller order queue, status updates
/api/reviews	Verified-purchase reviews per product
/api/admin	Seller approval, platform-wide stats and oversight

Each request passes through a `protect` middleware (verifies the JWT and loads the user) and, where needed, an `authorize(...roles)` middleware that rejects the request unless the user's role matches. Controllers stay thin and delegate data logic to Mongoose models.

7. Brand Value Propositions

Scalability

Stateless JWT authentication means any number of API instances can serve requests without shared session state. MongoDB Atlas scales independently, and the seller/order model (one document per seller per checkout) keeps per-seller queries fast as sellers and order volume grow.

Security

Passwords are hashed with bcrypt (never stored in plain text). Password-reset tokens are hashed before storage and expire in 30 minutes. Every sensitive route is protected by JWT verification plus explicit role checks, so a buyer token can never reach seller/admin-only endpoints.

Performance

Product search uses a MongoDB text index rather than scanning every document. Listings are server-side paginated (never loading the full catalog at once), and images are served from Cloudinary's CDN rather than the app's own server.

8. Tech Stack

Backend

- Node.js / Express.js
- MongoDB Atlas + Mongoose
- JSON Web Tokens (JWT) + bcrypt
- Stripe API (payments)

File & Media Handling

- Multer (in-memory upload handling)
- Cloudinary (image storage + CDN delivery)

Frontend

- React 18 (Vite)
- React Router, Tailwind CSS, Axios
- Stripe.js / React Stripe.js
- lucide-react (icons)

Development & Deployment Tools

- Git & GitHub for version control
- Vercel for both frontend and backend hosting (git-push-to-deploy)
- Nodemailer + SMTP for transactional email

Real-time communication (WebSockets/push notifications) and a formal CI/CD pipeline (automated tests, containerization) were scoped out of this version — see Future Improvements.

9. Challenges & Solutions

CHALLENGE — SPLITTING ORDERS ACROSS SELLERS

A single cart can contain products from different sellers, but each seller needs to manage only their own orders. **Solution:** on checkout, cart items are grouped by seller in-memory, and one `Order` document is created per seller, all sharing a common `orderGroupId`.

CHALLENGE — TRUSTING CLIENT-REPORTED PAYMENT SUCCESS

A malicious client could claim a payment succeeded without paying. **Solution:** the backend re-verifies the `PaymentIntent`'s status directly with Stripe's API before creating any order or decrementing stock.

CHALLENGE — SOURCING ACCURATE DEMO PRODUCT IMAGES

Early seed data used random stock-photo pools, which produced mismatched images. **Solution:** the seed script queries the Pexels image-search API live, using a clean search term per product name, so each demo product gets a genuinely matching photo.

CHALLENGE — KEEPING A GROWING UI PROFESSIONAL UNDER DEADLINE PRESSURE

As scope grew, it was easy for the UI to default to plain, unstyled markup. **Solution:** the interface was iterated deliberately — a distinct dark navbar, consistent brand color, hover/transition polish on product cards, and a proper footer.

10. Database Design

Five core Mongoose models, connected by ObjectId references:

- **User** — single collection for all three roles, distinguished by a `role` enum; sellers carry a nested `shopInfo` (shop name, approval status)
- **Product** — references its owning seller; carries a MongoDB text index across name/description/category for search
- **Cart** — one document per user (unique index on `user`), holding an array of product/quantity pairs
- **Order** — one document per seller per checkout, linked to sibling orders via `orderGroupId`; stores a price/name snapshot per item so later product edits never change historical order data
- **Review** — compound unique index on `(product, buyer)` so a buyer can only review a given product once

11. Application Flow

User (Buyer) Flow

Register/Login → Browse & search products → Add to cart → Checkout (shipping address + Stripe card payment) → Order(s) created and stock updated → Track order status → Leave a review once delivered

Seller Flow

Register with shop details → Await admin approval → (Once approved) List products with images → Receive orders containing their products → Update order status as it ships and is delivered

Admin Flow

Log in → Review pending seller applications → Approve or reject sellers → Monitor all products and orders platform-wide → Review aggregate stats (revenue, order volume, seller/buyer counts)

12. Best Practices

12.1 Authentication & Security

- JWT-based stateless authentication (no OAuth was implemented in this version)
- Passwords hashed with bcrypt; reset tokens hashed before storage and time-limited to 30 minutes
- Role-based middleware (`protect` + `authorize`) guarding every sensitive route

12.2 Component Architecture

- Reusable frontend components: `ProductCard` , `OrderStatusBadge` , `PasswordInput` , `ProtectedRoute` , `Pagination`
- Modular backend structure: routes → controllers → models, kept strictly separated so each layer stays testable on its own

12.3 Error Handling & User Experience

- Consistent JSON error shape (`{ success, message }`) across every API endpoint
- Frontend surfaces backend error messages inline near the relevant form/action instead of generic alerts
- Dedicated logging/monitoring tooling (e.g. Sentry) was not integrated in this version — noted as a future improvement

13. Results

- A fully functional multi-vendor marketplace with real payment processing, not a mock checkout
- Three working role-based dashboards (buyer shopping flow, seller product/order management, admin oversight)
- A seeded catalog of 52 realistic products across three categories, validating search, filtering, and pagination under realistic data volume
- A codebase organized into clear, independently testable phases (auth → products → cart/payments → admin/reviews → frontend → polish → deployment)

14. Lessons Learned

- Payment flows need server-side verification at every step — client-reported state is never sufficient for anything involving money
- Multi-vendor data models are meaningfully more complex than single-vendor ones; designing the Order schema around "one document per seller" early avoided a messier retrofit later
- Demo/seed data deserves the same care as production data — mismatched or broken demo content undermines trust in an otherwise solid build
- Incremental, phase-based delivery made a large scope manageable and easy to test along the way

15. Future Improvements

- Real-time buyer↔seller messaging and order notifications (WebSockets)
- Additional payment gateways beyond Stripe (e.g. PayPal, local rails)
- Per-seller sales analytics dashboard (charts over time, not just totals)
- Seller payout automation (currently, payments settle to a single platform Stripe account)
- Automated CI/CD pipeline and test coverage (unit + integration tests) for the API layer
- Dedicated logging/monitoring (e.g. Sentry) for production error visibility

16. Links

LIVE APPLICATION

<https://shopease-ecommerce-one.vercel.app/>

GITHUB REPOSITORY

<https://github.com/Haroonkhan4063/shopease-ecommerce>